Лабораторная работа №1

Задание 1.

Параметр nIndex: COLOR_BTNFACE, COLOR_GRAYTEXT, COLOR_DESKTOP

Код и результат:

```cpp
#include <iostream>
#include <Windows.h>

int main() {
    COLORREF btnFaceColor = GetSysColor(COLOR_BTNFACE);
    COLORREF grayTextColor = GetSysColor(COLOR_GRAYTEXT);
    COLORREF desktopColor = GetSysColor(COLOR_DESKTOP);

    std::cout << "COLOR_BTNFACE: " << btnFaceColor << std::endl;
    std::cout << "COLOR_GRAYTEXT: " << grayTextColor << std::endl;
    std::cout << "COLOR_DESKTOP: " << desktopColor << std::endl;

    return 0;
}
```

```
Консоль отладки Microsoft Visual Studio
COLOR_BTNFACE: 15790320
COLOR_GRAYTEXT: 7171437
COLOR_DESKTOP: 0
```

Задание 2.

Название API-функции: GetLocalTime, GetTimeZoneInformation, EnuinDate Formats

Код и результат:

```cpp
#include <iostream>
#include <Windows.h>
#include <locale.h>

BOOL CALLBACK EnumDateFormatsProc(LPTSTR lpDateFormatString) {
    std::wcout << lpDateFormatString << std::endl;
    return TRUE;
}

int main() {
    SYSTEMTIME sysTime;
    GetLocalTime(&sysTime);
    std::cout << "Локальное время: " << sysTime.wHour << ":" << sysTime.wMinute << ":" << sysTime.wSecond << std::endl;

    std::cout << "Формат даты: " << std::endl;
    EnumDateFormats(EnumDateFormatsProc, MAKELCID(MAKELANGID(LANG_ENGLISH, SUBLANG_ENGLISH_US), SORT_DEFAULT), DATE_SHORTDATE);
    return 0;
}
```

```
Консоль отладки Microsoft Visual Studio
Локальное время: 13:54:20
Формат даты:
M/d/yyyy
M/d/yy
MM/dd/yy
MM/dd/yyyy
yy/MM/dd
yyyy-MM-dd
dd-MMM-yy
M/d/yyyy
M/d/yy
MM/dd/yy
MM/dd/yyyy
```

Задание 3.

Название API-функции: AnsiToOemBuff, GetCursorPos, GetNumberFormat, SetCaretPos

Код и результат:

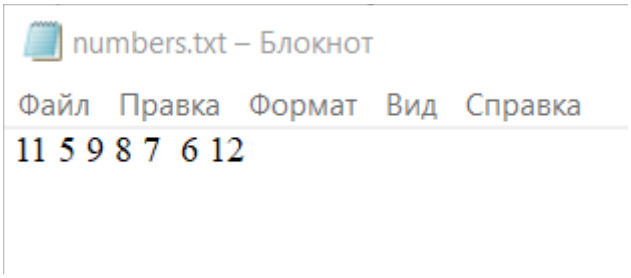
```cpp
#include <iostream>
#include <Windows.h>

int main() {
    char inputBuffer[] = "Пример текста";
    char outputBuffer[100];
    AnsiToOemBuff(inputBuffer, outputBuffer, sizeof(inputBuffer));

    std::cout << "Преобразованный текст: " << outputBuffer << std::endl;

    POINT cursorPos;
    GetCursorPos(&cursorPos);
    std::cout << "Позиция курсора: X= " << cursorPos.x << ", Y= " << cursorPos.y << std::endl;

    double number = 12345.6789;
    wchar_t formattedNumber[20];
    GetNumberFormatW(LOCALE_USER_DEFAULT, 0, std::to_wstring(number).c_str(), nullptr, formattedNumber, sizeof(formattedNumber) / sizeof(formattedNumber[0]));

    std::wcout << L"Форматированное число: " << formattedNumber << std::endl;

    SetCaretPos(100, 100);
    std::cout << "Позиция установлена на (100, 100)." << std::endl;
    return 0;
}
```

Лабораторная работа №2

Задание 1.

Найти минимальное число в файле, содержащем

числа.



```cpp
#include <windows.h>
#include <iostream>
#include <fstream>
#include <vector>
#include <algorithm>

int main() {
    // Создание текстового файла и запись чисел в него
    std::ofstream file("numbers.txt");
    if (!file.is_open()) {
        std::cerr << "Failed to create file." << std::endl;
        return 1;
    }
    file << "11 5 9 8 7 6 12";
    file.close();
    std::cout << "Numbers written to file successfully." << std::endl;

    // Открытие файла и поиск минимального числа
    std::ifstream inputFile("numbers.txt");
    if (!inputFile.is_open()) {
        std::cerr << "Failed to open file." << std::endl;
        return 1;
    }
```

```cpp
    std::vector<int> numbers;
    int num;
    while (inputFile >> num) {
        numbers.push_back(num);
    }
    inputFile.close();

    if (numbers.empty()) {
        std::cerr << "No numbers found in file." << std::endl;
        return 1;
    }

    int minNumber = *std::min_element(numbers.begin(), numbers.end());

    std::cout << "The minimum number in the file is: " << minNumber << std::endl;

    return 0;
}
```

```
Numbers written to file successfully.
The minimum number in the file is: 5
```

Лабораторная работа №3

Функции: GetMemoryStatus, ListProcesses.

```cpp
#include <windows.h>
#include <tlhelp32.h>
#include <psapi.h>
#include <iostream>
#include <vector>

void GetMemoryStatus() {
    MEMORYSTATUS memStatus;
    GlobalMemoryStatus(&memStatus);

    std::cout << "Total physical memory: " << memStatus.dwTotalPhys / (1024 * 1024) << " MB" << std::endl;
    std::cout << "Available physical memory: " << memStatus.dwAvailPhys / (1024 * 1024) << " MB" << std::endl;
    std::cout << "Total virtual memory: " << memStatus.dwTotalVirtual / (1024 * 1024) << " MB" << std::endl;
    std::cout << "Available virtual memory: " << memStatus.dwAvailVirtual / (1024 * 1024) << " MB" << std::endl;
}

void ListProcesses() {
    HANDLE hProcessSnap;
    PROCESSENTRY32 pe32;
    hProcessSnap = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);

    if (hProcessSnap == INVALID_HANDLE_VALUE) {
        std::cerr << "CreateToolhelp32Snapshot failed!" << std::endl;
        return;
    }

    pe32.dwSize = sizeof(PROCESSENTRY32);
    if (!Process32First(hProcessSnap, &pe32)) {
        std::cerr << "Process32First failed!" << std::endl;
        CloseHandle(hProcessSnap);
        return;
    }
```

```cpp
    do {
        std::wcout << L"Process name: " << pe32.szExeFile << L" (PID: " << pe32.th32ProcessID << L")" << std::endl;

        HANDLE hProcess = OpenProcess(PROCESS_QUERY_INFORMATION | PROCESS_VM_READ, FALSE, pe32.th32ProcessID);
        if (hProcess != NULL) {
            PROCESS_MEMORY_COUNTERS pmc;
            if (GetProcessMemoryInfo(hProcess, &pmc, sizeof(pmc))) {
                std::cout << "    WorkingSetSize: " << pmc.WorkingSetSize / 1024 << " KB" << std::endl;
                std::cout << "    PagefileUsage: " << pmc.PagefileUsage / 1024 << " KB" << std::endl;
            }
            CloseHandle(hProcess);
        }
    } while (Process32Next(hProcessSnap, &pe32));

    CloseHandle(hProcessSnap);
}

int main() {
    GetMemoryStatus();
    ListProcesses();
    return 0;
}
```

```
Total physical memory: 7890 MB
Available physical memory: 951 MB
Total virtual memory: 134217727 MB
Available virtual memory: 134213582 MB
Process name: [System Process] (PID: 0)
Process name: System (PID: 4)
Process name: Secure System (PID: 72)
Process name: Registry (PID: 128)
Process name: smss.exe (PID: 584)
Process name: csrss.exe (PID: 948)
Process name: wininit.exe (PID: 772)
Process name: services.exe (PID: 1064)
Process name: LsaIso.exe (PID: 1084)
Process name: lsass.exe (PID: 1092)
Process name: svchost.exe (PID: 1212)
Process name: fontdrvhost.exe (PID: 1236)
Process name: WUDFHost.exe (PID: 1280)
Process name: svchost.exe (PID: 1392)
Process name: svchost.exe (PID: 1472)
Process name: WUDFHost.exe (PID: 1668)
Process name: svchost.exe (PID: 1716)
Process name: svchost.exe (PID: 1788)
Process name: svchost.exe (PID: 1808)
Process name: svchost.exe (PID: 1848)
Process name: svchost.exe (PID: 1856)
Process name: svchost.exe (PID: 1868)
Process name: svchost.exe (PID: 2008)
Process name: svchost.exe (PID: 1156)
Process name: svchost.exe (PID: 1104)
Process name: svchost.exe (PID: 2124)
```

Лабораторная работа №4

```cpp
#include <windows.h>
#include <tlhelp32.h>
#include <psapi.h>
#include <iostream>

// Функция для получения идентификатора текущего процесса
DWORD getCurrentProcessId() {
    return GetCurrentProcessId();
}

// Функция для получения псевдодескриптора текущего процесса
HANDLE getCurrentProcessHandle() {
    return GetCurrentProcess();
}

// Функция для дублирования дескриптора
HANDLE duplicateProcessHandle(HANDLE hProcess) {
    HANDLE hDuplicate = NULL;
    if (DuplicateHandle(GetCurrentProcess(), hProcess, GetCurrentProcess(), &hDuplicate, 0, FALSE, DUPLICATE_SAME_ACCESS)) {
        return hDuplicate;
    }
    else {
        std::cerr << "Failed to duplicate handle. Error: " << GetLastError() << std::endl;
        return NULL;
    }
}
```

```cpp
// Функция для открытия процесса по его идентификатору
HANDLE openProcess(DWORD processId) {
    HANDLE hProcess = OpenProcess(PROCESS_QUERY_INFORMATION | PROCESS_VM_READ, FALSE, processId);
    if (hProcess == NULL) {
        std::cerr << "Failed to open process. Error: " << GetLastError() << std::endl;
    }
    return hProcess;
}

// Функция для закрытия дескриптора
void closeHandle(HANDLE hProcess) {
    if (!CloseHandle(hProcess)) {
        std::cerr << "Failed to close handle. Error: " << GetLastError() << std::endl;
    }
}

// Функция для создания снимка процессов
HANDLE createSnapshot() {
    HANDLE hSnapshot = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
    if (hSnapshot == INVALID_HANDLE_VALUE) {
        std::cerr << "Failed to create snapshot. Error: " << GetLastError() << std::endl;
    }
    return hSnapshot;
}
```

```cpp
// Функция для вывода списка процессов
void listProcesses() {
    HANDLE hSnapshot = createSnapshot();
    if (hSnapshot == INVALID_HANDLE_VALUE) {
        return;
    }

    PROCESSENTRY32 pe32;
    pe32.dwSize = sizeof(PROCESSENTRY32);

    if (!Process32First(hSnapshot, &pe32)) {
        std::cerr << "Failed to get first process. Error: " << GetLastError() << std::endl;
        CloseHandle(hSnapshot);
        return;
    }

    do {
        std::wcout << L"Process Name: " << pe32.szExeFile << L" | Process ID: " << pe32.th32ProcessID << std::endl;
    } while (Process32Next(hSnapshot, &pe32));

    CloseHandle(hSnapshot);
}
```

```cpp
// Функция для получения информации о памяти процесса
void getProcessMemoryInfo(DWORD processId) {
    HANDLE hProcess = openProcess(processId);
    if (hProcess == NULL) {
        return;
    }

    PROCESS_MEMORY_COUNTERS pmc;
    if (GetProcessMemoryInfo(hProcess, &pmc, sizeof(pmc))) {
        std::cout << "Process ID: " << processId << std::endl;
        std::cout << "Working Set Size: " << pmc.WorkingSetSize << std::endl;
        std::cout << "Pagefile Usage: " << pmc.PagefileUsage << std::endl;
    }
    else {
        std::cerr << "Failed to get memory info for process with ID: " << processId << std::endl;
    }

    closeHandle(hProcess);
}
```

```cpp
int main() {
    // Шаг 1: Получение идентификатора текущего процесса
    DWORD processId = getCurrentProcessId();
    std::cout << "Current Process ID: " << processId << std::endl;

    // Шаг 2: Получение псевдодескриптора текущего процесса
    HANDLE hProcess = getCurrentProcessHandle();
    std::cout << "Current Process Handle: " << hProcess << std::endl;

    // Шаг 3: Дублирование дескриптора
    HANDLE hDuplicate = duplicateProcessHandle(hProcess);
    if (hDuplicate != NULL) {
        std::cout << "Duplicate Process Handle: " << hDuplicate << std::endl;

        // Шаг 4: Открытие процесса
        HANDLE hOpenProcess = openProcess(processId);
        if (hOpenProcess != NULL) {
            std::cout << "Open Process Handle: " << hOpenProcess << std::endl;

            // Шаг 5: Закрытие дублированного дескриптора
            closeHandle(hDuplicate);
            std::cout << "Closed Duplicate Handle" << std::endl;

            // Шаг 6: Закрытие дескриптора открытого процесса
            closeHandle(hOpenProcess);
            std::cout << "Closed Open Process Handle" << std::endl;
        }
    }
```

```cpp
    // Вывод списка процессов и их информации
    listProcesses();

    // Пример получения информации о памяти текущего процесса
    getProcessMemoryInfo(processId);

    return 0;
}
```

```
Process Name: Lenovo.Modern.ImController.PluginHost.Device.exe | Process ID: 16728
Process Name: browser.exe | Process ID: 21636
Process Name: vcpkgsrv.exe | Process ID: 17812
Process Name: browser.exe | Process ID: 21940
Process Name: browser.exe | Process ID: 9184
Process Name: browser.exe | Process ID: 21164
Process Name: browser.exe | Process ID: 7572
Process Name: vcpkgsrv.exe | Process ID: 22452
Process Name: vcpkgsrv.exe | Process ID: 20704
Process Name: RuntimeBroker.exe | Process ID: 5052
Process Name: backgroundTaskHost.exe | Process ID: 10188
Process Name: RuntimeBroker.exe | Process ID: 21124
Process Name: msedgewebview2.exe | Process ID: 17872
Process Name: msedgewebview2.exe | Process ID: 17644
Process Name: msedgewebview2.exe | Process ID: 1816
Process Name: msedgewebview2.exe | Process ID: 6092
Process Name: msedgewebview2.exe | Process ID: 1704
Process Name: WmiPrvSE.exe | Process ID: 7000
Process Name: VsDebugConsole.exe | Process ID: 20684
Process Name: conhost.exe | Process ID: 11652
Process Name: LR.exe | Process ID: 13104
Process Name: msvsmon.exe | Process ID: 3108
Process ID: 13104
Working Set Size: 3928064
Pagefile Usage: 794624
```

Лабораторная работа №5

Имитация снятия денег со счета несколькими клиентами для наличия положительного баланса.

```cpp
#include <iostream>
#include <thread>
#include <mutex>

using namespace std;

class BankAccount {
private:
    int balance;
    mutex mtx;

public:
    BankAccount(int initialBalance) : balance(initialBalance) {}

    void takeMoney(int amount) {
        lock_guard<mutex> lock(mtx);
        if (balance >= amount) {
            balance -= amount;
            cout << "Thread " << this_thread::get_id() << " withdrew " << amount << " dollars. Balance: " << balance << endl;
        }
        else {
            cout << "Thread " << this_thread::get_id() << " tried to withdraw " << amount << " dollars, but insufficient funds." << endl;
        }
    }

    int checkBalance() {
        return balance;
    }
};
```

```cpp
class Client {
private:
    BankAccount& account;
    int amount;

public:
    Client(BankAccount& acc, int amt) : account(acc), amount(amt) {}

    void operator()() {
        while (account.checkBalance() > 0) {
            account.takeMoney(amount);
            this_thread::sleep_for(chrono::milliseconds(100)); // Pause to simulate time taken to withdraw money
        }
    }
};

int main() {
    int initialBalance = 1000; // Initial balance
    const int numberOfClients = 5; // Number of clients
    int withdrawalAmount = 100; // Amount to withdraw by each client

    BankAccount account(initialBalance);

    cout << "Running with synchronization:" << endl;

    thread clients[numberOfClients];
    for (int i = 0; i < numberOfClients; ++i) {
        clients[i] = thread(Client(account, withdrawalAmount));
    }
```

```cpp
    for (int i = 0; i < numberOfClients; ++i) {
        clients[i].join();
    }

    return 0;
}
```

Результат:

```
Running with synchronization:
Thread 2552 withdrew 100 dollars. Balance: 900
Thread 14648 withdrew 100 dollars. Balance: 800
Thread 2436 withdrew 100 dollars. Balance: 700
Thread 22492 withdrew 100 dollars. Balance: 600
Thread 21120 withdrew 100 dollars. Balance: 500
Thread 14648 withdrew 100 dollars. Balance: 400
Thread 2552 withdrew 100 dollars. Balance: 300
Thread 21120 withdrew 100 dollars. Balance: 200
Thread 22492 withdrew 100 dollars. Balance: 100
Thread 2436 withdrew 100 dollars. Balance: 0
```